



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	TE-AI&DS	Semester:	V
Course Code:	2015111	Course Name:	Statistics for Machine Learning and Data Science Lab

Name of Student:	Anish Mangesh Naik
Roll No. :	79
Experiment No.:	7
Title of the Experiment:	Build and Evaluate Classification Models using Supervised Learning Algorithms
Date of Performance:	
Date of Submission:	

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty :
Signature :
Date :



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Aim: To build supervised classification models for predicting diabetes outcomes using the Pima Indians Diabetes Dataset and evaluate their performance using appropriate classification metrics.

Objectives

After completing this experiment, the student will be able to:

1. Build supervised classification models to predict the diabetes outcome of patients.
2. Evaluate and interpret classification model performance using appropriate evaluation metrics.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical information for **768 female patients**.

The target variable is:

- Outcome = 0: Non-diabetic
- Outcome = 1: Diabetic



Procedure

1. Load the Pima Indians Diabetes Dataset.
2. Separate the input features and target variable.
3. Identify and handle invalid or missing values.
4. Divide the dataset into training and testing sets using a stratified split.
5. Apply suitable preprocessing, such as feature scaling where required.
6. Build one or more classification models, such as:
 - Logistic Regression
 - k-Nearest Neighbors
 - Decision Tree
7. Train the selected model(s) using the training data.
8. Generate predictions for the test data.
9. Construct the confusion matrix.
10. Calculate classification performance metrics.
11. Compare the obtained results and interpret the model performance.

Description of the Experiment

Classification is a supervised learning technique used to assign observations to predefined categories.

In this experiment, the objective is to classify a patient as: Patient Health Data, Classification Model gives Non-Diabetic and Diabetic.

The experiment follows: Dataset, Feature and Target Selection, Data Preprocessing, Train-Test Split, Classification Model, Prediction, and Performance Evaluation



Detailed Description of Statistical Methods

Logistic Regression

Logistic regression predicts the probability of a binary outcome.

The logistic function is:

$$P(Y = 1) = \frac{1}{1+e^{-z}}$$

where:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

The predicted probability is converted into a class using a decision threshold.

Confusion Matrix

A confusion matrix contains:

Predicted Negative Predicted Positive

Actual Negative True Negative (TN) False Positive (FP)

Actual Positive False Negative (FN) True Positive (TP)

Accuracy

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision

$$Precision = \frac{TP}{TP+FP}$$

Precision indicates how many predicted positive cases were actually positive.

Recall

$$Recall = \frac{TP}{TP+FN}$$



Recall indicates how many actual positive cases were correctly identified.

F1-Score

$$F1 = 2 \left(\frac{Precision \times Recall}{Precision + Recall} \right)$$

The F1-score balances precision and recall.

ROC-AUC

The ROC curve represents the relationship between:

- True Positive Rate
- False Positive Rate

The Area Under the Curve (AUC) provides an overall measure of the model's ability to distinguish between the two classes.

Code:-

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.impute import SimpleImputer
```

```
from sklearn.linear_model import LogisticRegression
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import (
```

```
    confusion_matrix,
```

```
    accuracy_score,
```

```
    precision_score,
```

```
    recall_score,
```

```
    f1_score,
```

```
    roc_auc_score,
```

```
    roc_curve,
```

```
    classification_report
```

```
)
```

```
df = pd.read_csv("diabetes.csv")
```

```
print("First 5 rows of the dataset:")
```

```
print(df.head())
```

```
print("\nDataset Shape:")
```

```
print(df.shape)
```



```
print("\nDataset Information:")

print(df.info())

print("\nMissing Values:")

print(df.isnull().sum())


print("\nNumber of Zero Values:")

print((df == 0).sum())

invalid_columns = [

    "Glucose",

    "BloodPressure",

    "SkinThickness",

    "Insulin",

    "BMI"

]

df[invalid_columns] = df[invalid_columns].replace(0, np.nan)


print("\nMissing Values After Replacing Invalid Zeros:")

print(df.isnull().sum())

X = df.drop("Outcome", axis=1)

y = df["Outcome"]
```



```
print("\nFeature Shape:", X.shape)

print("Target Shape:", y.shape)

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.20,

    random_state=42,

    stratify=y

)

print("\nTraining Samples:", X_train.shape[0])

print("Testing Samples:", X_test.shape[0])

print("\nTraining Class Distribution:")

print(y_train.value_counts())

print("\nTesting Class Distribution:")

print(y_test.value_counts())

imputer = SimpleImputer(strategy="median")

X_train_imputed = imputer.fit_transform(X_train)
```



```
X_test_imputed = imputer.transform(X_test)
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train_imputed)
```

```
X_test_scaled = scaler.transform(X_test_imputed)
```

```
logistic_model = LogisticRegression(
```

```
    max_iter=1000,
```

```
    random_state=42
```

```
)
```

```
knn_model = KNeighborsClassifier(
```

```
    n_neighbors=5
```

```
)
```

```
decision_tree_model = DecisionTreeClassifier(
```

```
    random_state=42,
```

```
    max_depth=5
```

```
)
```



```
logistic_model.fit(X_train_scaled, y_train)
```

```
y_pred_lr = logistic_model.predict(X_test_scaled)
```

```
y_prob_lr = logistic_model.predict_proba(X_test_scaled)[:, 1]
```

```
knn_model.fit(X_train_scaled, y_train)
```

```
y_pred_knn = knn_model.predict(X_test_scaled)
```

```
y_prob_knn = knn_model.predict_proba(X_test_scaled)[:, 1]
```

```
decision_tree_model.fit(X_train_imputed, y_train)
```

```
y_pred_dt = decision_tree_model.predict(X_test_imputed)
```

```
y_prob_dt = decision_tree_model.predict_proba(X_test_imputed)[:, 1]
```

```
def evaluate_model(model_name, y_true, y_pred, y_prob):
```

```
    cm = confusion_matrix(y_true, y_pred)
```

```
    accuracy = accuracy_score(y_true, y_pred)
```

```
    precision = precision_score(
```

```
        y_true,
```

```
        y_pred,
```

```
        zero_division=0
```

```
    )
```



```
recall = recall_score(
```

```
    y_true,
```

```
    y_pred,
```

```
    zero_division=0
```

```
)
```

```
f1 = f1_score(
```

```
    y_true,
```

```
    y_pred,
```

```
    zero_division=0
```

```
)
```

```
auc = roc_auc_score(y_true, y_prob)
```

```
print("\n" + "=" * 60)
```

```
print(model_name)
```

```
print("=" * 60)
```

```
print("\nConfusion Matrix:")
```

```
print(cm)
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
print("\nAccuracy :", round(accuracy, 4))

print("Precision:", round(precision, 4))

print("Recall  :", round(recall, 4))

print("F1-Score :", round(f1, 4))

print("ROC-AUC  :", round(auc, 4))


print("\nClassification Report:")

print(classification_report(

    y_true,

    y_pred,

    target_names=["Non-Diabetic", "Diabetic"],

    zero_division=0

))


return {

    "Model": model_name,

    "Accuracy": accuracy,

    "Precision": precision,

    "Recall": recall,

    "F1-Score": f1,

    "ROC-AUC": auc
```



```
}  
  
results = []  
  
results.append(  
    evaluate_model(  
        "Logistic Regression",  
        y_test,  
        y_pred_lr,  
        y_prob_lr  
    )  
)  
  
results.append(  
    evaluate_model(  
        "K-Nearest Neighbors",  
        y_test,  
        y_pred_knn,  
        y_prob_knn  
    )  
)
```



```
results.append(  
    evaluate_model(  
        "Decision Tree",  
        y_test,  
        y_pred_dt,  
        y_prob_dt  
    )  
)  
  
results_df = pd.DataFrame(results)  
  
print("\n\nMODEL PERFORMANCE COMPARISON")  
  
print("=" * 80)  
  
print(  
    results_df.round(4).to_string(index=False)  
)  
  
metrics = [  
    "Accuracy",  
    "Precision",  
    "Recall",  
    "F1-Score",
```



"ROC-AUC"

]

```
results_plot = results_df.set_index("Model")[metrics]
```

```
results_plot.plot(
```

```
    kind="bar",
```

```
    figsize=(12, 6)
```

```
)
```

```
plt.title("Comparison of Classification Models")
```

```
plt.ylabel("Score")
```

```
plt.xlabel("Classification Model")
```

```
plt.ylim(0, 1)
```

```
plt.xticks(rotation=0)
```

```
plt.legend(loc="lower right")
```

```
plt.grid(axis="y", alpha=0.3)
```

```
plt.show()
```



```
cm_lr = confusion_matrix(y_test, y_pred_lr)
```

```
plt.figure(figsize=(6, 5))
```

```
sns.heatmap(  
    cm_lr,  
    annot=True,  
    fmt="d",  
    cmap="Blues",  
    xticklabels=["Non-Diabetic", "Diabetic"],  
    yticklabels=["Non-Diabetic", "Diabetic"]  
)
```

```
plt.title("Confusion Matrix - Logistic Regression")
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.show()
```

```
cm_knn = confusion_matrix(y_test, y_pred_knn)
```

```
plt.figure(figsize=(6, 5))
```



```
sns.heatmap(  
  
    cm_knn,  
  
    annot=True,  
  
    fmt="d",  
  
    cmap="Greens",  
  
    xticklabels=["Non-Diabetic", "Diabetic"],  
  
    yticklabels=["Non-Diabetic", "Diabetic"]  
  
)
```

```
plt.title("Confusion Matrix - KNN")  
  
plt.xlabel("Predicted")  
  
plt.ylabel("Actual")
```

```
plt.show()  
  
cm_dt = confusion_matrix(y_test, y_pred_dt)
```

```
plt.figure(figsize=(6, 5))
```

```
sns.heatmap(  
  
    cm_dt,
```



```
annot=True,  
  
fmt="d",  
  
cmap="Oranges",  
  
xticklabels=["Non-Diabetic", "Diabetic"],  
  
yticklabels=["Non-Diabetic", "Diabetic"]  
  
)  
  
  
plt.title("Confusion Matrix - Decision Tree")  
  
plt.xlabel("Predicted")  
  
plt.ylabel("Actual")  
  
  
  
plt.show()  
  
fpr_lr, tpr_lr, _ = roc_curve(y_test, y_prob_lr)  
  
fpr_knn, tpr_knn, _ = roc_curve(y_test, y_prob_knn)  
  
fpr_dt, tpr_dt, _ = roc_curve(y_test, y_prob_dt)  
  
  
plt.figure(figsize=(8, 6))  
  
  
plt.plot(  
  
    fpr_lr,  
  
    tpr_lr,
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
label=f"Logistic Regression (AUC = {roc_auc_score(y_test, y_prob_lr):.3f})"
)
```

```
plt.plot(
    fpr_knn,
    tpr_knn,
    label=f"KNN (AUC = {roc_auc_score(y_test, y_prob_knn):.3f})"
)
```

```
plt.plot(
    fpr_dt,
    tpr_dt,
    label=f"Decision Tree (AUC = {roc_auc_score(y_test, y_prob_dt):.3f})"
)
```

```
plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curves for Classification Models")


plt.legend()

plt.grid(alpha=0.3)


plt.show()

best_model = results_df.loc[

    results_df["ROC-AUC"].idxmax()

]


print("\nBEST MODEL BASED ON ROC-AUC")

print("=" * 50)


print("Model   :", best_model["Model"])

print("Accuracy :", round(best_model["Accuracy"], 4))

print("Precision:", round(best_model["Precision"], 4))

print("Recall   :", round(best_model["Recall"], 4))

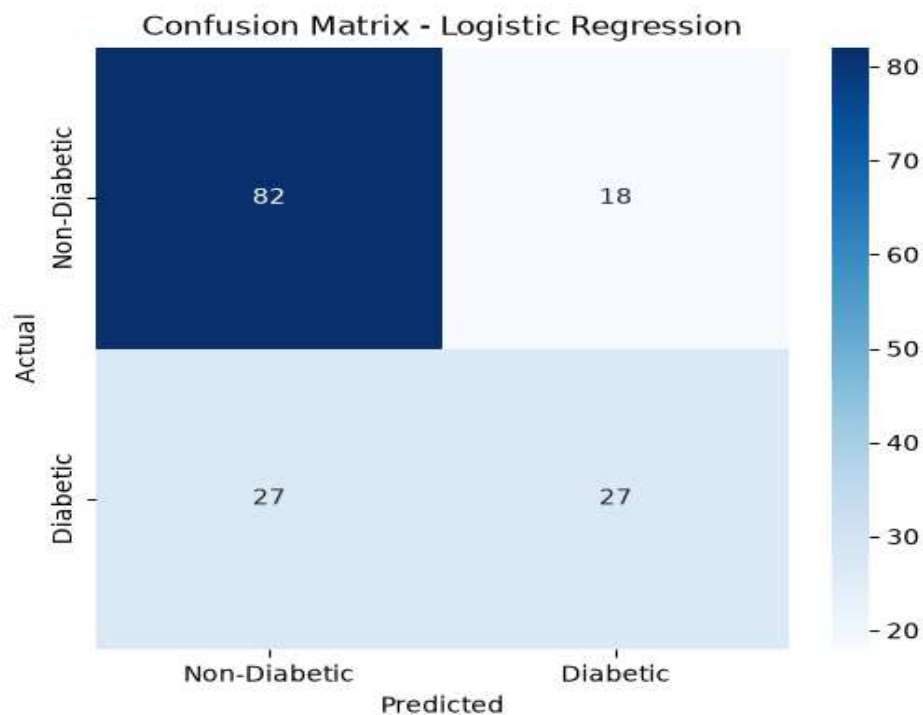
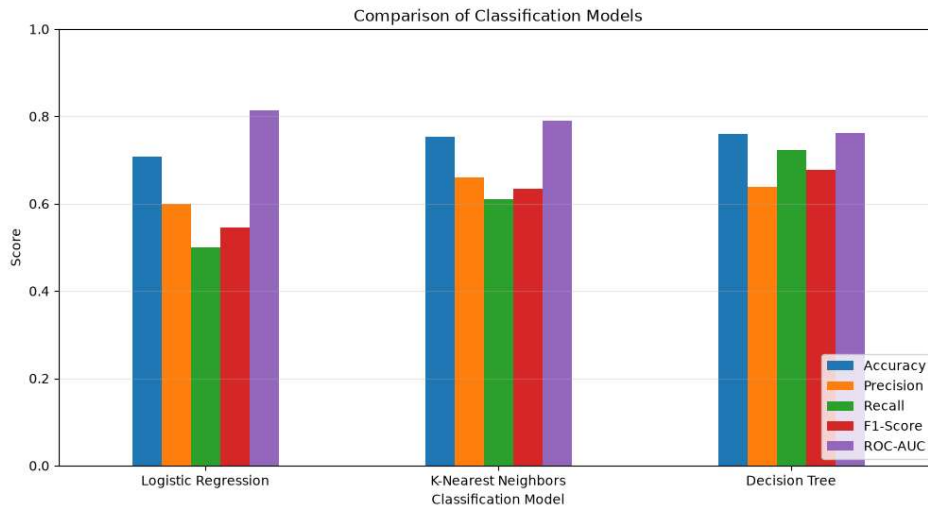
print("F1-Score :", round(best_model["F1-Score"], 4))

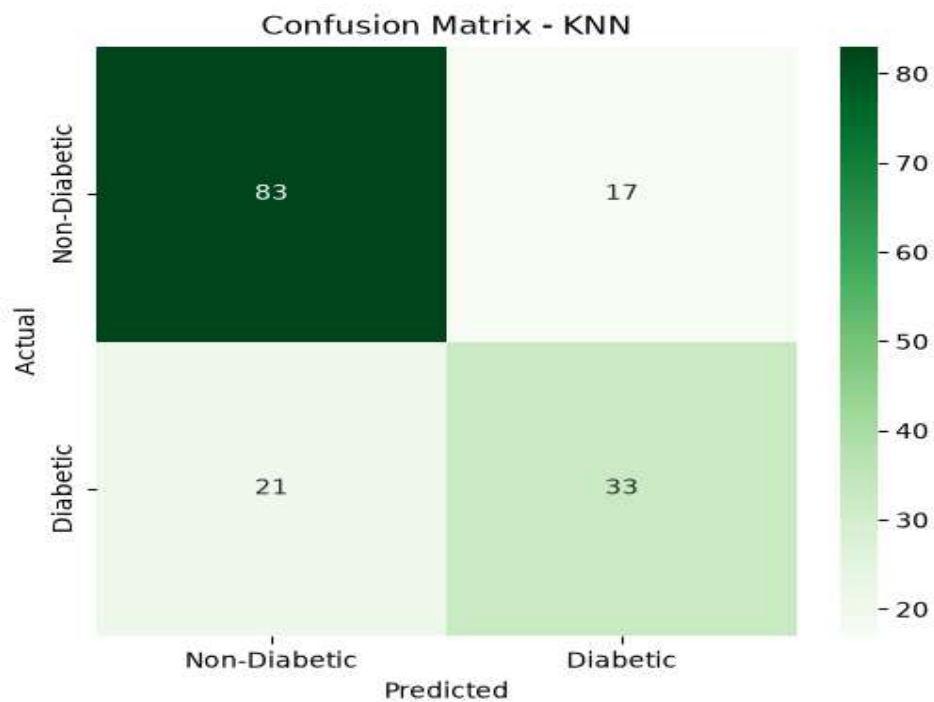
2015111: Statistics for Machine Learning and Data Science Lab
```

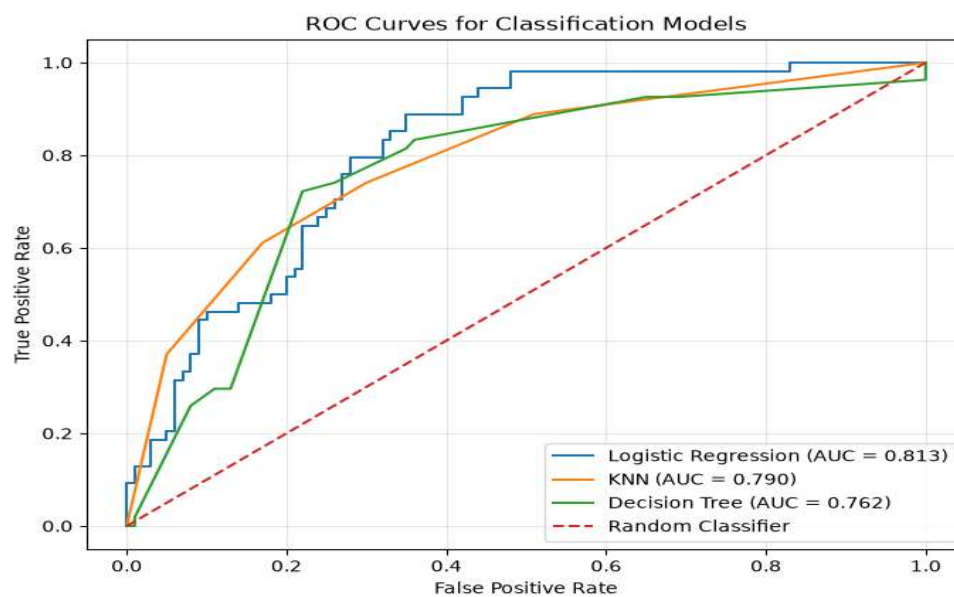
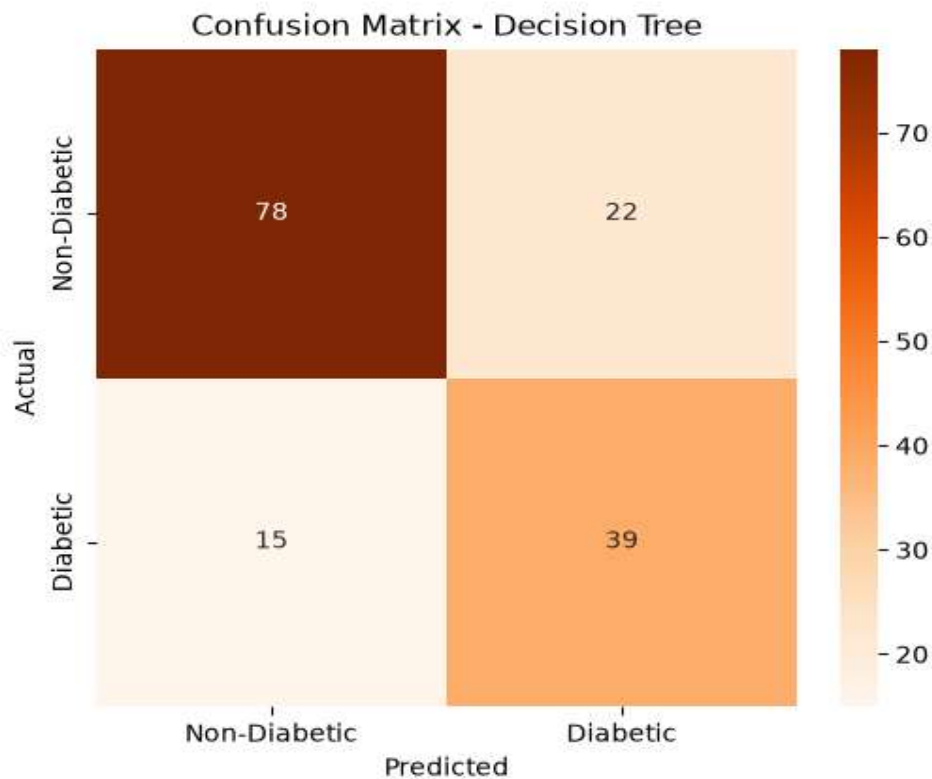


```
print("ROC-AUC :", round(best_model["ROC-AUC"], 4))
```

Output:-









Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

First 5 rows of the dataset:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Dataset Shape:

(768, 9)

Dataset Information:

<class 'pandas.DataFrame'>

RangeIndex: 768 entries, 0 to 767

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Pregnancies	768 non-null	int64
1	Glucose	768 non-null	int64
2	BloodPressure	768 non-null	int64
3	SkinThickness	768 non-null	int64
4	Insulin	768 non-null	int64
5	BMI	768 non-null	float64
6	DiabetesPedigreeFunction	768 non-null	float64
7	Age	768 non-null	int64
8	Outcome	768 non-null	int64

dtypes: float64(2), int64(7)

memory usage: 54.1 KB

None

Missing Values:

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

Number of Zero Values:

Pregnancies	111
Glucose	5
BloodPressure	35
SkinThickness	227



```
Number of Zero Values:
Pregnancies      111
Glucose           5
BloodPressure    35
SkinThickness    227
Insulin          374
BMI              11
DiabetesPedigreeFunction  0
Age              0
Outcome          500
dtype: int64

Missing Values After Replacing Invalid Zeros:
Pregnancies      0
Glucose           5
BloodPressure    35
SkinThickness    227
Insulin          374
BMI              11
DiabetesPedigreeFunction  0
Age              0
Outcome          0
dtype: int64

Feature Shape: (768, 8)
Target Shape: (768,)

Training Samples: 614
Testing Samples: 154

Training Class Distribution:
Outcome
0    400
1    214
Name: count, dtype: int64

Testing Class Distribution:
Outcome
0    100
1     54
Name: count, dtype: int64
```



Logistic Regression

Confusion Matrix:

```
[[82 18]
 [27 27]]
```

Accuracy : 0.7078

Precision: 0.6

Recall : 0.5

F1-Score : 0.5455

ROC-AUC : 0.813

Classification Report:

	precision	recall	f1-score	support
Non-Diabetic	0.75	0.82	0.78	100
Diabetic	0.60	0.50	0.55	54
accuracy			0.71	154
macro avg	0.68	0.66	0.67	154
weighted avg	0.70	0.71	0.70	154

K-Nearest Neighbors

Confusion Matrix:

```
[[83 17]
 [21 33]]
```

Accuracy : 0.7532

Precision: 0.66

Recall : 0.6111

F1-Score : 0.6346

ROC-AUC : 0.7899

Classification Report:

	precision	recall	f1-score	support
Non-Diabetic	0.80	0.83	0.81	100
Diabetic	0.66	0.61	0.63	54
accuracy			0.75	154
macro avg	0.73	0.72	0.72	154
weighted avg	0.75	0.75	0.75	154



Decision Tree

Confusion Matrix:

```
[[78 22]
 [15 39]]
```

Accuracy : 0.7597

Precision: 0.6393

Recall : 0.7222

F1-Score : 0.6783

ROC-AUC : 0.7622

Classification Report:

	precision	recall	f1-score	support
Non-Diabetic	0.84	0.78	0.81	100
Diabetic	0.64	0.72	0.68	54
accuracy			0.76	154
macro avg	0.74	0.75	0.74	154
weighted avg	0.77	0.76	0.76	154

MODEL PERFORMANCE COMPARISON

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.7078	0.6000	0.5000	0.5455	0.8130
K-Nearest Neighbors	0.7532	0.6600	0.6111	0.6346	0.7899
Decision Tree	0.7597	0.6393	0.7222	0.6783	0.7622

BEST MODEL BASED ON ROC-AUC

Model : Logistic Regression

Accuracy : 0.7078

Precision: 0.6

Recall : 0.5

F1-Score : 0.5455

ROC-AUC : 0.813

PS C:\Users\admin\Documents\New folder>

Conclusion

Supervised classification models were developed to predict diabetes outcomes using the Pima Indians Diabetes Dataset. The models were evaluated using the confusion matrix, accuracy,



precision, recall, F1-score, and ROC-AUC. The experiment demonstrated that different evaluation metrics provide different perspectives on classification performance.

Questions to be Answered by Students

1.Which classification model was implemented, and what were its accuracy, precision, recall, and F1-score?

Three classification models were implemented: Logistic Regression, K-Nearest Neighbors (KNN), and Decision Tree.

For Logistic Regression:

- Accuracy: 0.7078 (70.78%)
- Precision: 0.6000 (60.00%)
- Recall: 0.5000 (50.00%)
- F1-score: 0.5455 (54.55%)

2.How many true positives and false negatives were obtained from the confusion matrix?

The Logistic Regression confusion matrix is:

[[82 18]

[27 27]]

Therefore:

- True Positive (TP) = 27
- False Negative (FN) = 27

So, 27 diabetic patients were correctly identified, while 27 diabetic patients were incorrectly classified as non-diabetic.



3.Which metric is more important for diabetes prediction: precision or recall? Justify your answer.

Recall is more important for diabetes prediction.

Recall measures how many actual diabetic patients are correctly identified. A false negative means a patient who actually has diabetes is predicted as non-diabetic. Missing such patients can delay diagnosis and treatment.

Therefore, a model with higher recall is generally preferred for diabetes screening.

In your results, the Decision Tree has the highest recall (0.7222) among the three models.

4.What was the ROC-AUC value, and what does it indicate about the model's performance?

For Logistic Regression:

ROC-AUC = 0.813

This indicates that the Logistic Regression model has good discriminatory ability to distinguish between diabetic and non-diabetic patients.

An AUC of 0.5 represents random classification, while an AUC closer to 1.0 represents better classification performance. Therefore, 0.813 indicates good overall classification performance.

Interestingly, Logistic Regression also has the highest ROC-AUC among your three models:

1. Logistic Regression: 0.813
2. KNN: 0.7899
3. Decision Tree: 0.7622

5.State two limitations of the developed classification model.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Limited dataset size: The model was trained on only 614 training samples from the 768-patient dataset, so its performance may not generalize perfectly to other populations.

Missing/invalid values: Several medical attributes contained zero values that were treated as invalid and replaced using median imputation. This may introduce some approximation and affect model predictions.